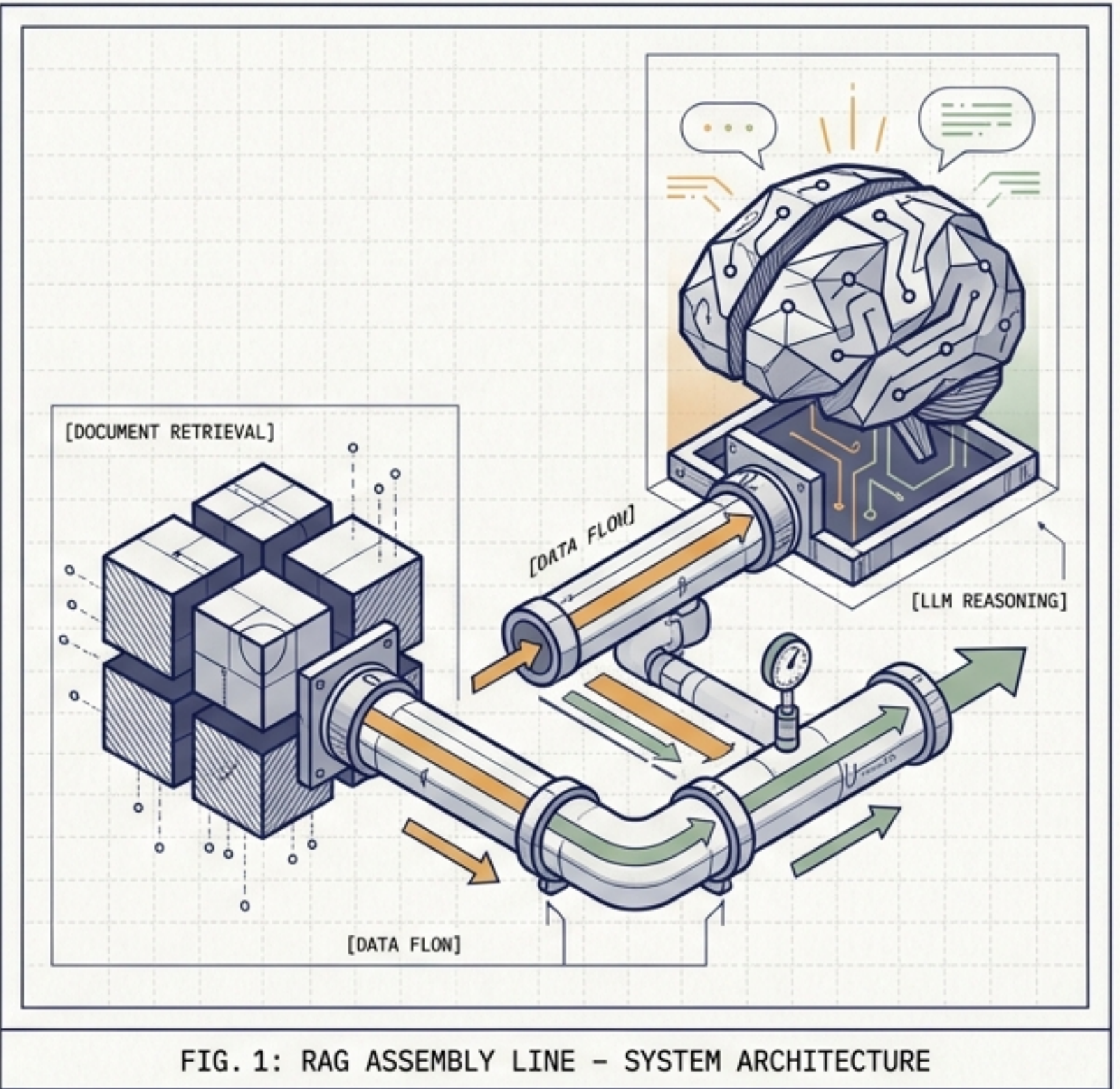


RAG Post-Retrieval: Dominando los Prompts en LangChain

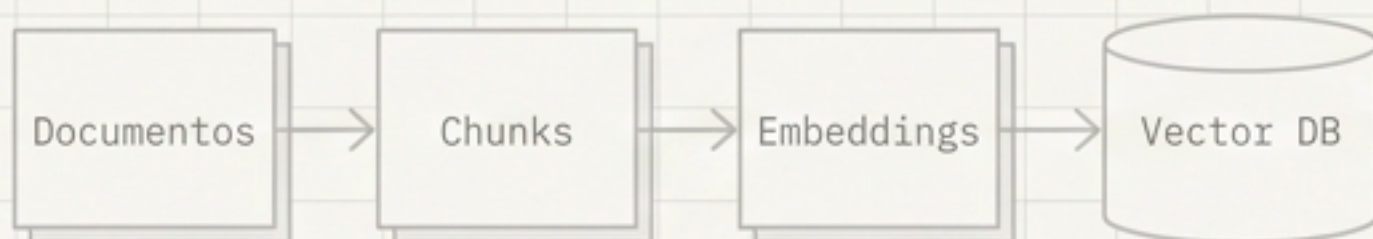
De documentos en bruto a respuestas generativas: Plantillas, Contexto y LCEL.

[NIVEL: INTERMEDIO]

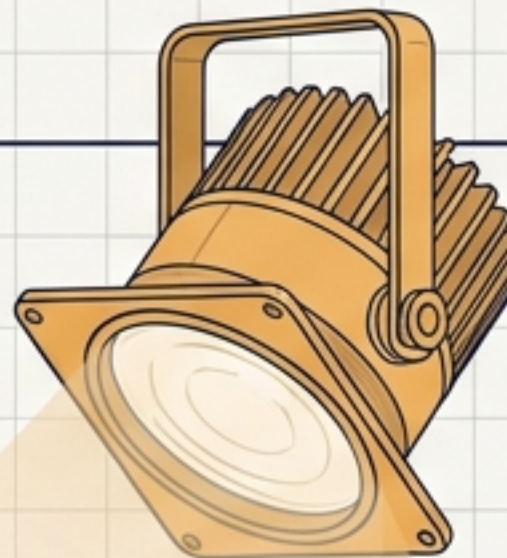
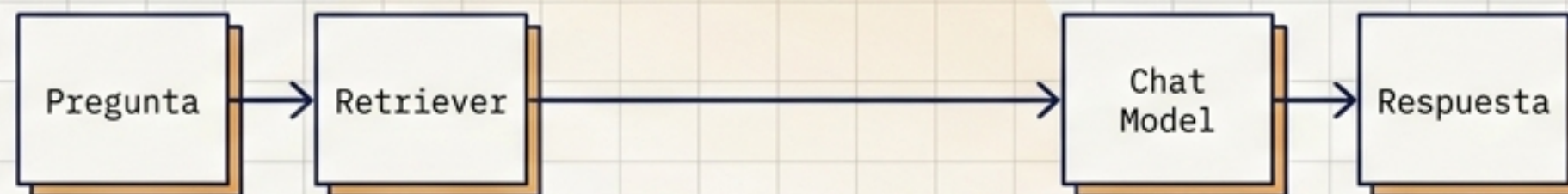
[ENFOQUE: ARQUITECTURA VISUAL]



Pipeline de Indexación



Pipeline de Consulta



El Mapa del RAG

- Ya recuperaste los Document[] correctos.
- La calidad del RAG se define aquí: ¿cómo le entregas estos datos al modelo para que no falle?

Un retriever excelente con un prompt pobre siempre generará respuestas mediocres.

Separación de Responsabilidades



Retriever

Rol: Encontrar

Responsabilidad: Extraer información relevante de la **base de datos**.



Formatter

Rol: Preparar

Responsabilidad: Convertir objetos `Document[]` en contexto legible.



Prompt Template

Rol: Ensamblar

Responsabilidad: Construir instrucciones, inyectar variables y definir reglas.



Chat Model

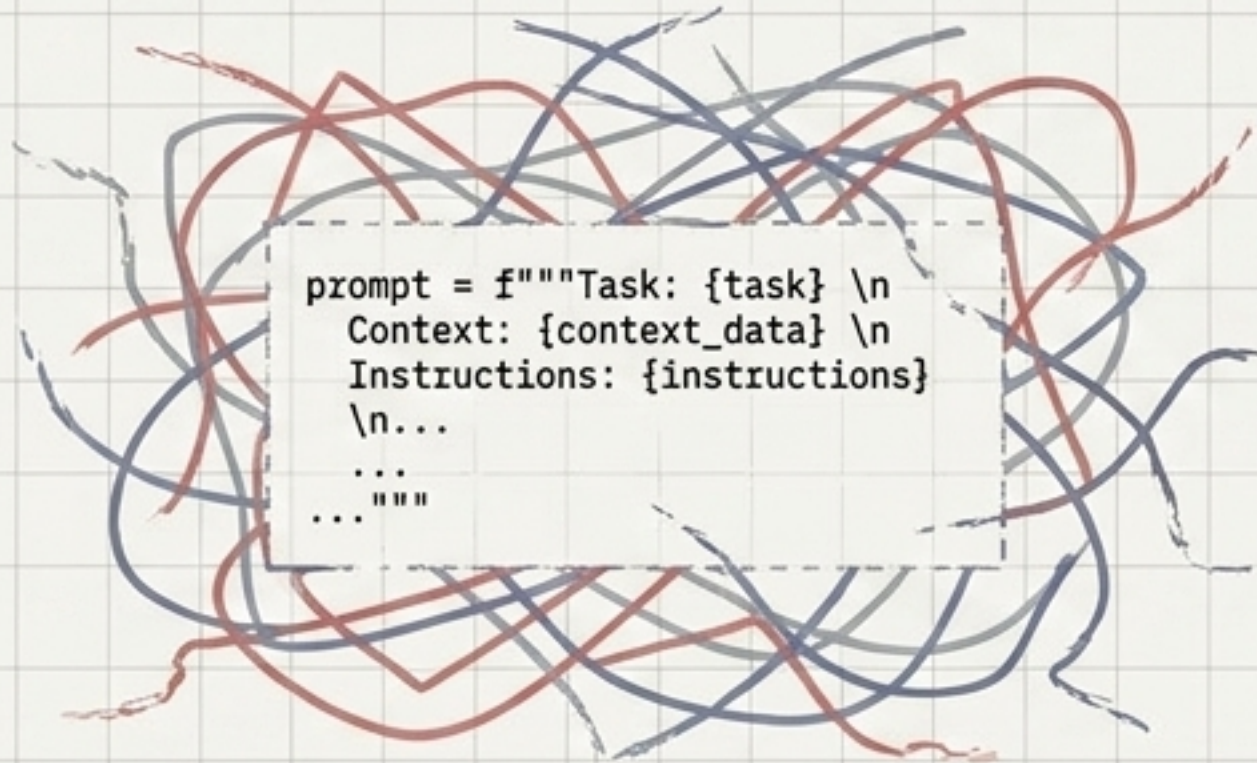
Rol: Sintetizar

Responsabilidad: Generar la respuesta final en lenguaje natural.

REGLA DE ORO: El retriever no responde, solo busca. El modelo no busca, solo genera.

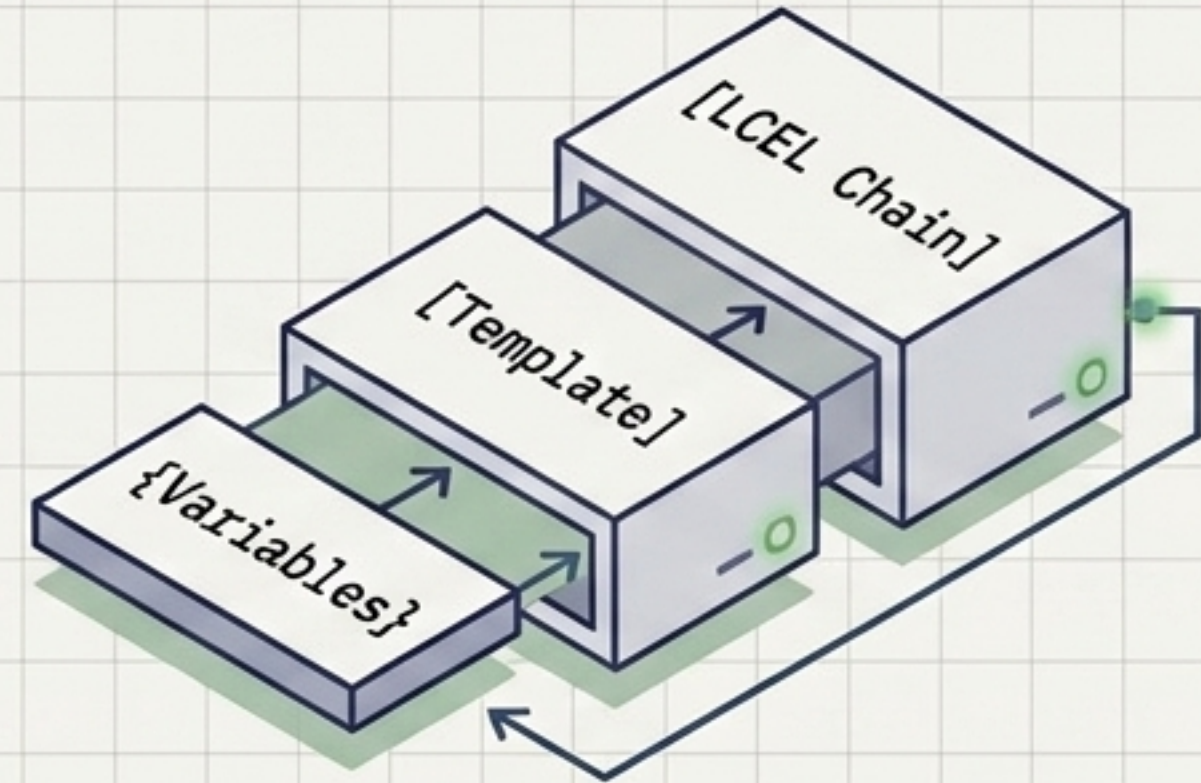
¿Por qué usar Prompt Templates?

Construcción Manual (f-strings)



- Frágil ante cambios
- Texto estático
- Difícil de componer
- Mezcla de roles

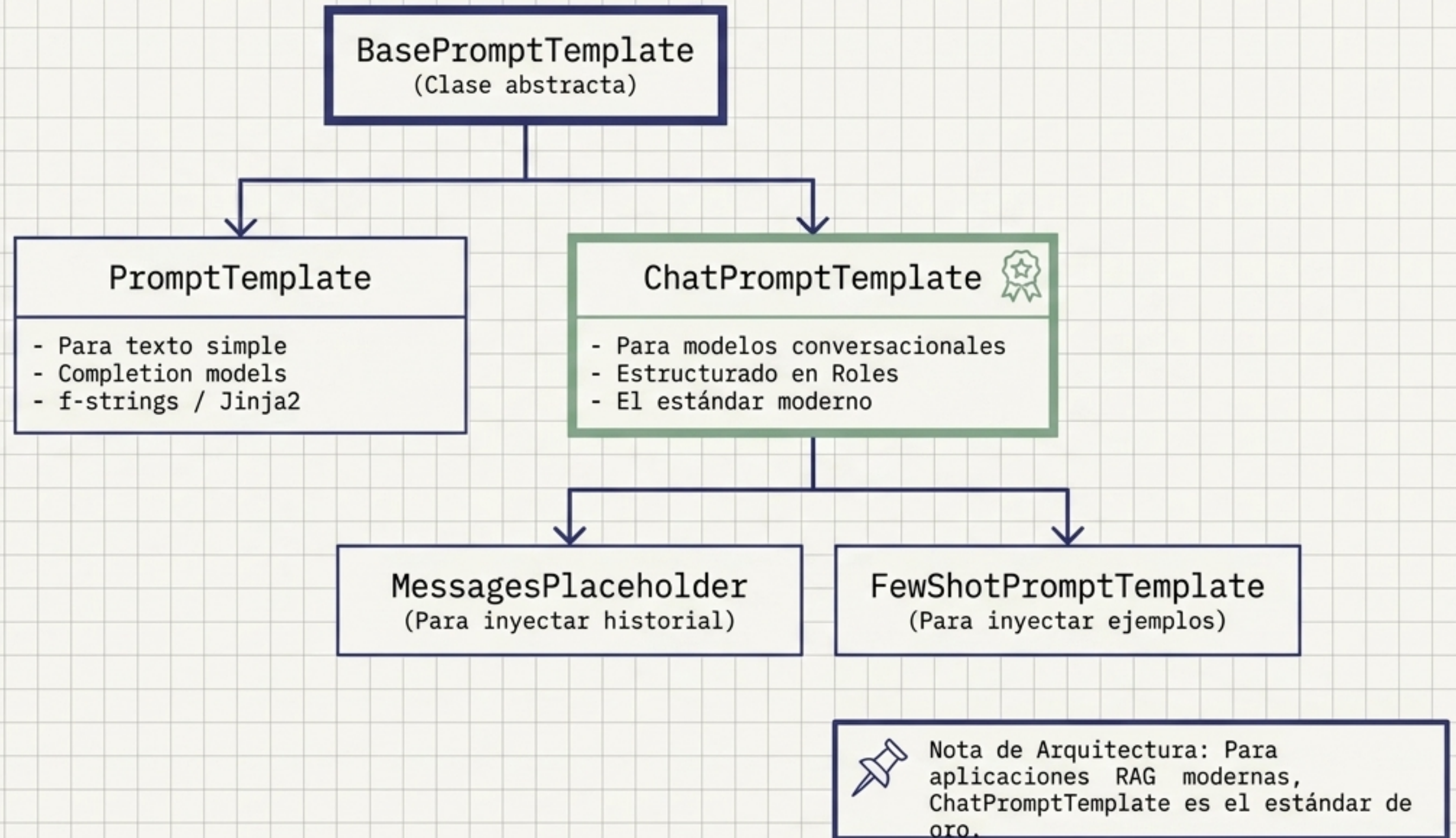
LangChain Templates



- Altamente reutilizable
- Validación integrada
- Manejo nativo de variables
- Compatible con interfaces Runnable

Los templates convierten los prompts de simples cadenas de texto a componentes estructurales de la aplicación.

La Jerarquía de Prompts en LangChain



ChatPromptTemplate y la Estructura de Roles

	System (Reglas)	Identidad, políticas operativas, y formato de salida.	Eres un experto en...
	Human (Inputs)	Contexto dinámico recuperado + Pregunta actual del usuario.	Pregunta: {question}
	AI (Ejemplos)	Simulaciones de respuestas previas para alinear el comportamiento.	Según el documento, la

Sintaxis Recomendada (Tuplas)

```
[  
  ("system", "Eres un asistente estricto.  
  Usa este contexto: {context}"),  
  ("human", "{question}")  
]
```


MessagesPlaceholder: Gestionando el Historial



Error común: variable {history} de texto plano

User
Hola

AI
¡Hola! ¿Cómo
puedo ayudarte?

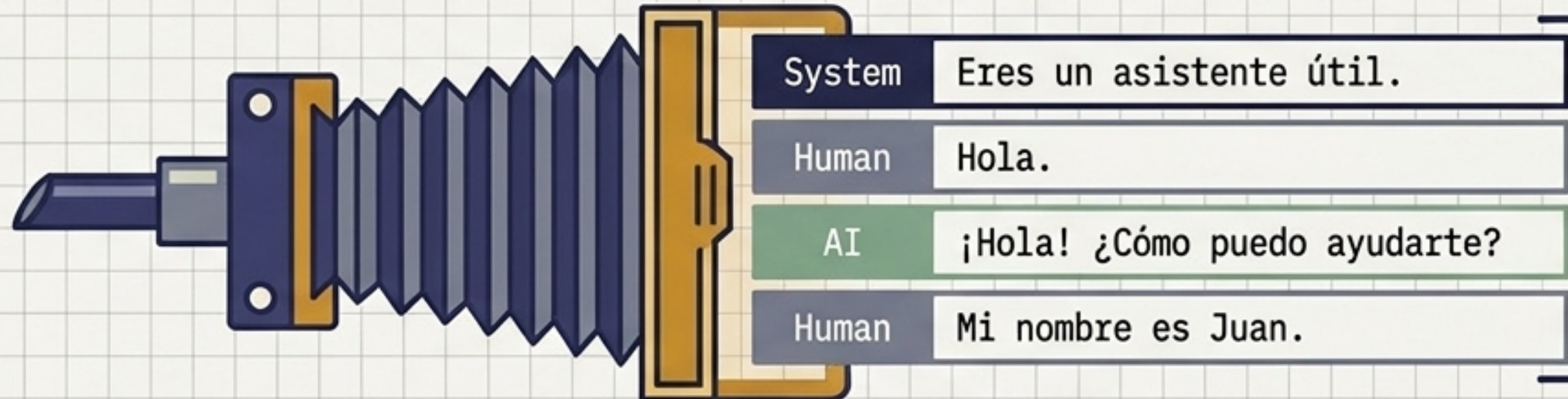
User
Mi nombre
es Juan.

Hola.
¡Hola!
¿Cómo puedo ayudarte?
Mi nombre es Juan.

Pierde la semántica
de quién dijo qué.
Es solo un bloque
de texto ciego.



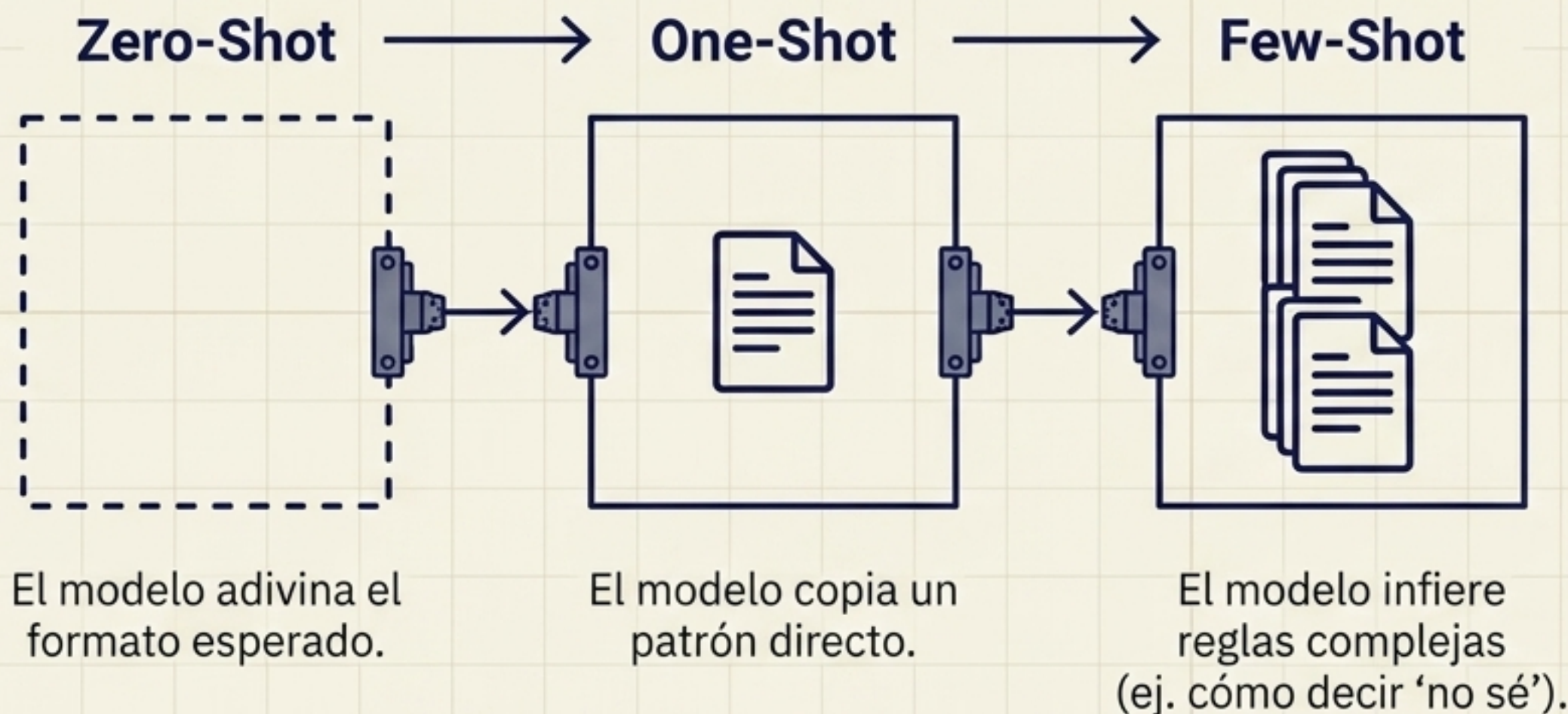
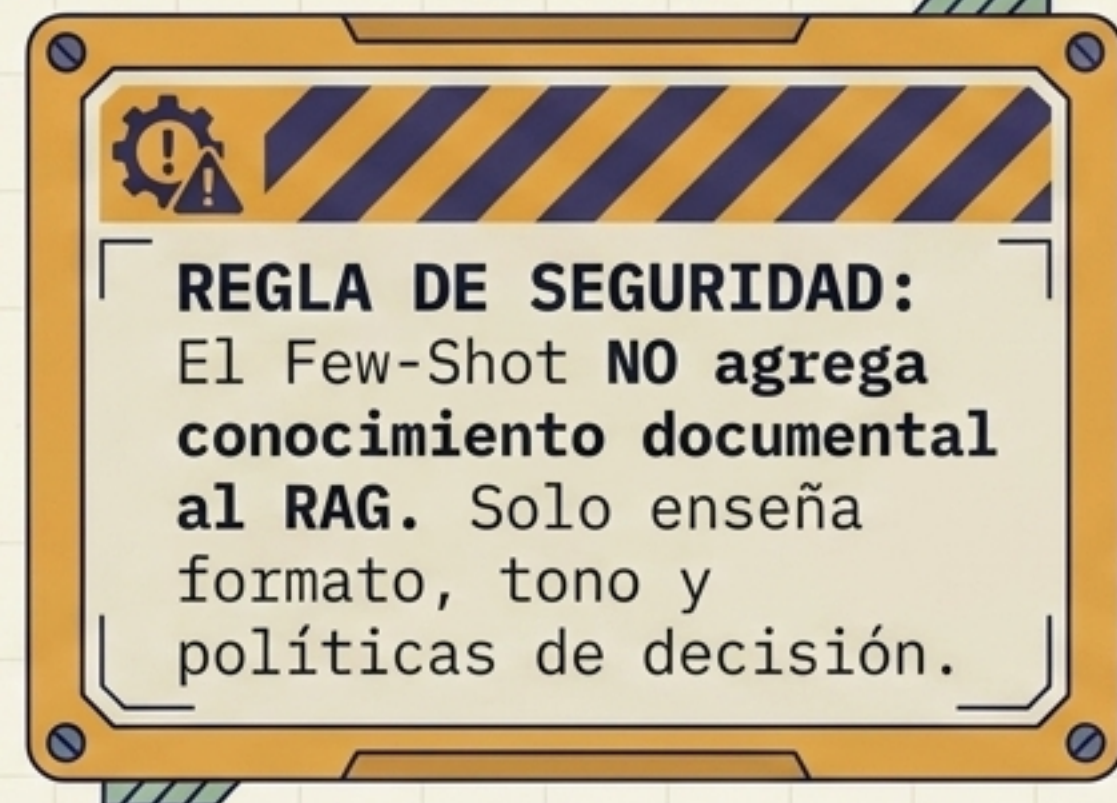
Solución: MessagesPlaceholder(variable_name='history')



Conserva la
semántica
estructural. El
modelo entiende el
flujo real de la
conversación.

Un string es solo texto. Un Placeholder preserva la arquitectura conversacional.

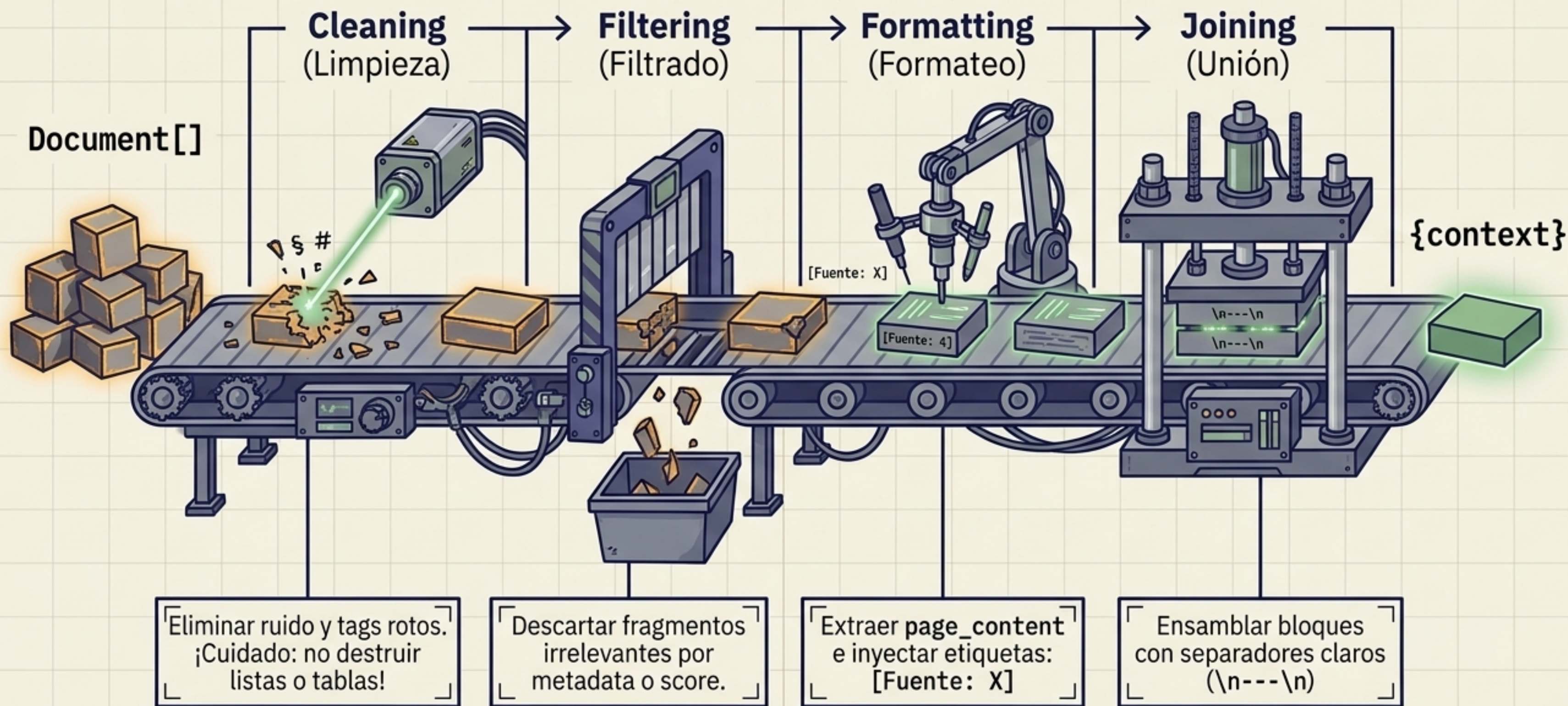
Enseñanza por Ejemplos (Few-Shot Prompting)



Example Selectors



El Puente de Contexto: De Document[] a Texto



Los 4 Pilares del Diseño de un Prompt RAG

1. System Prompt (La Ley)

¿Cómo debe comportarse el modelo?
(Rol, restricciones, formato y qué hacer si no hay evidencia).

2. Context (La Evidencia)

¿Qué información objetiva e inmutable recuperó el retriever?

3. Question (La Demanda)

¿Qué necesita el usuario resolver exactamente en este instante?

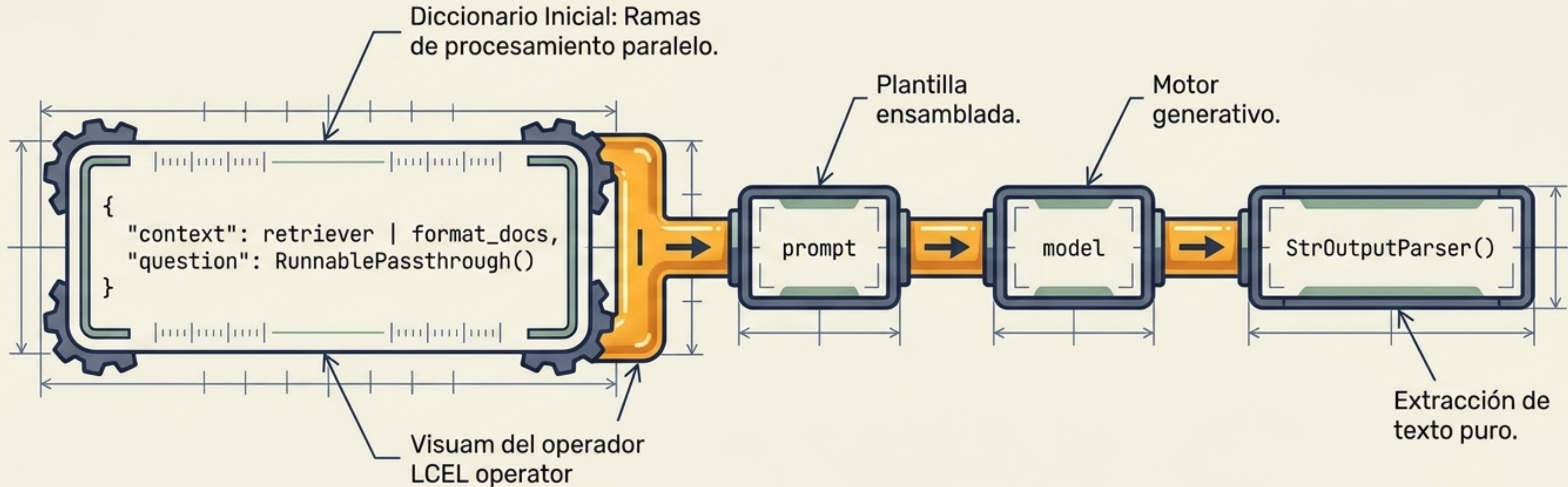
4. History (El Rastro)

¿Qué ocurrió antes en esta sesión para dar continuidad a la charla?



ADVERTENCIA DE SEGURIDAD: EL CONTEXTO SON DATOS, NO INSTRUCCIONES. Delimita la evidencia claramente (ej. `<context>...</context>`) para mitigar ataques de Prompt Injection.

LCEL: Ensamblando la Línea de Producción

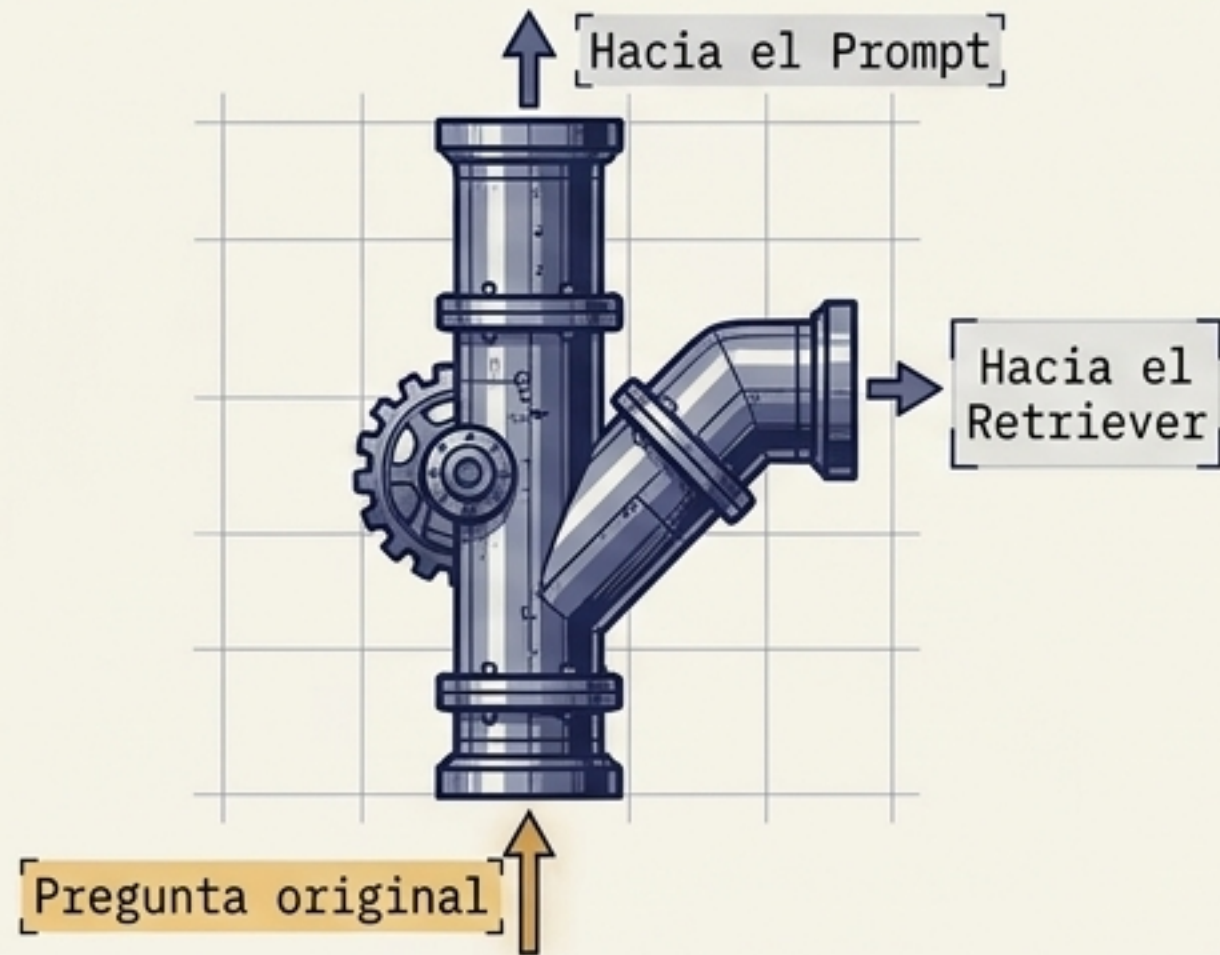


En LCEL, el operador **pipe (|)** significa que el output exacto de la izquierda se convierte en el input de la derecha. De datos crudos a texto generado en una sola línea de ensamblaje.

Disecccionando LCEL: El Divisor y el Embudo

RunnablePassthrough (El Divisor)

A

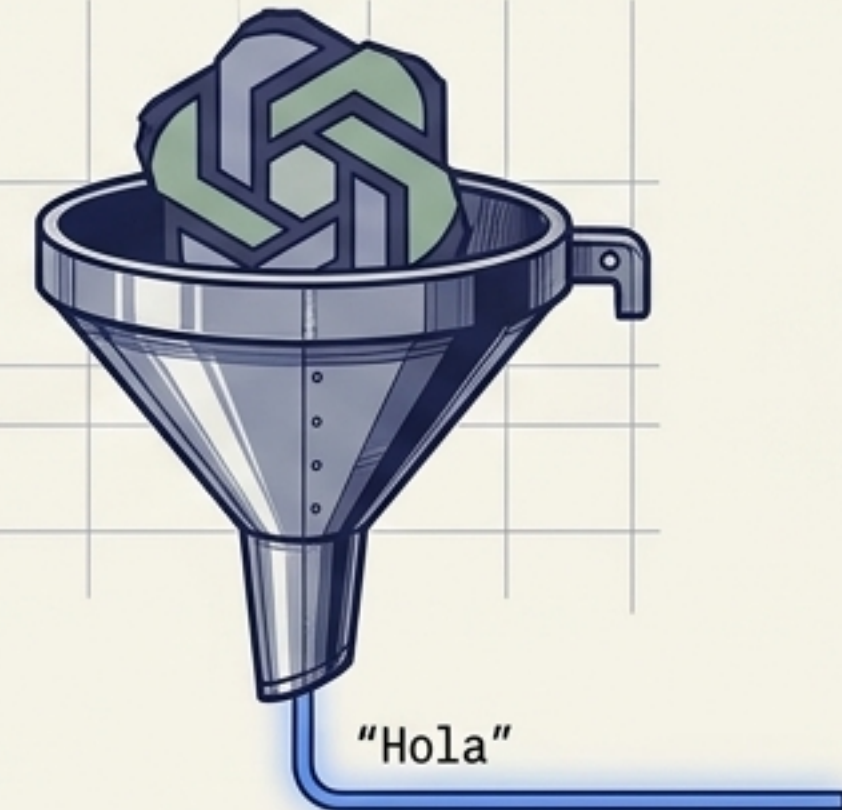


Permite que el input original 'pase' inalterado hacia adelante mientras simultáneamente alimenta ramas de procesamiento paralelo.

StrOutputParser (El Embudo)

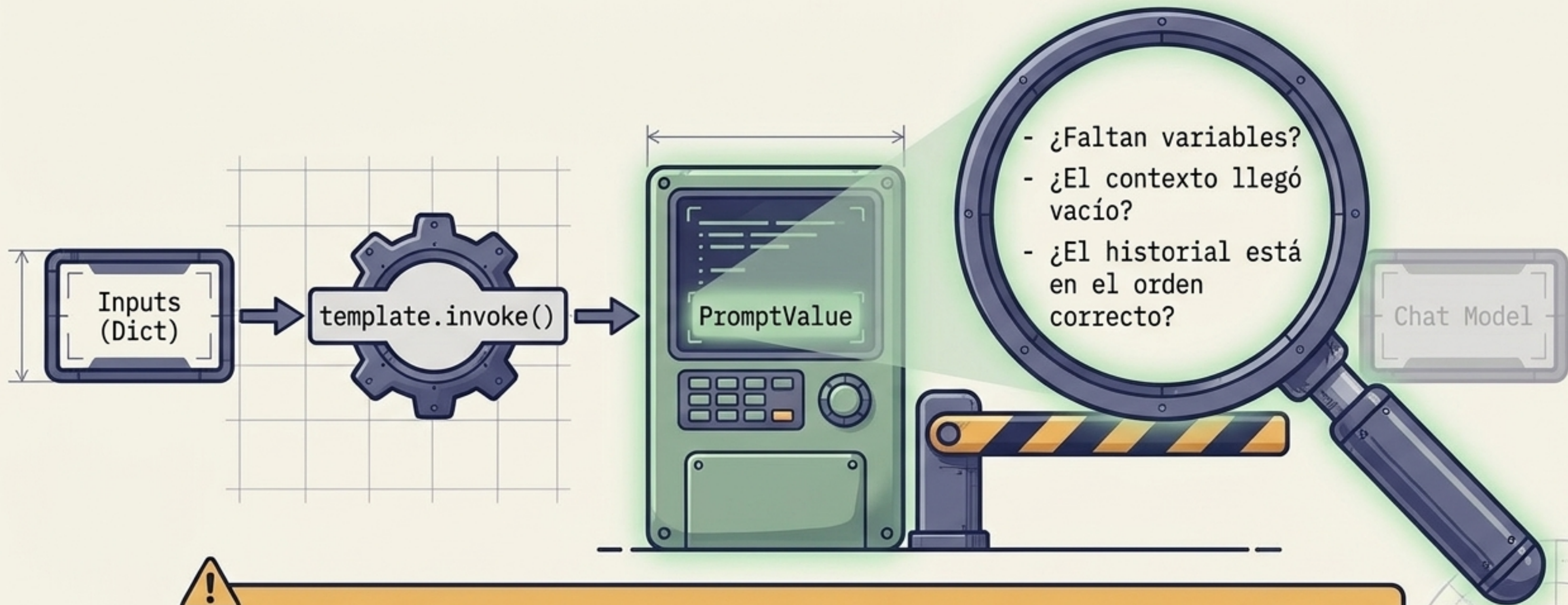
B

[AIMessage(content="Hola", metadata=...)]



Aísla y extrae el texto final limpio de la compleja estructura de respuesta que devuelve el modelo.

Debugging y el Valor del PromptValue



¡PUNTO DE CONTROL! Nunca llames al modelo en etapa de desarrollo sin inspeccionar el prompt compilado primero. Imprimir el PromptValue es el paso de debugging más valioso.

Los Pecados Capitaes del Post-Retrieval

MITO



Más contexto recuperado siempre es mejor para el modelo.

REALIDAD



Maximiza la **densidad de información**, no el volumen. Más chunks inútiles equivalen a **ruido** y **pérdida de atención**.

MITO



El historial conversacional sirve como fuente de verdad documental.

REALIDAD



Historial = continuidad temporal de la charla.
Documentos recuperados = única base de conocimiento autorizada.

MITO



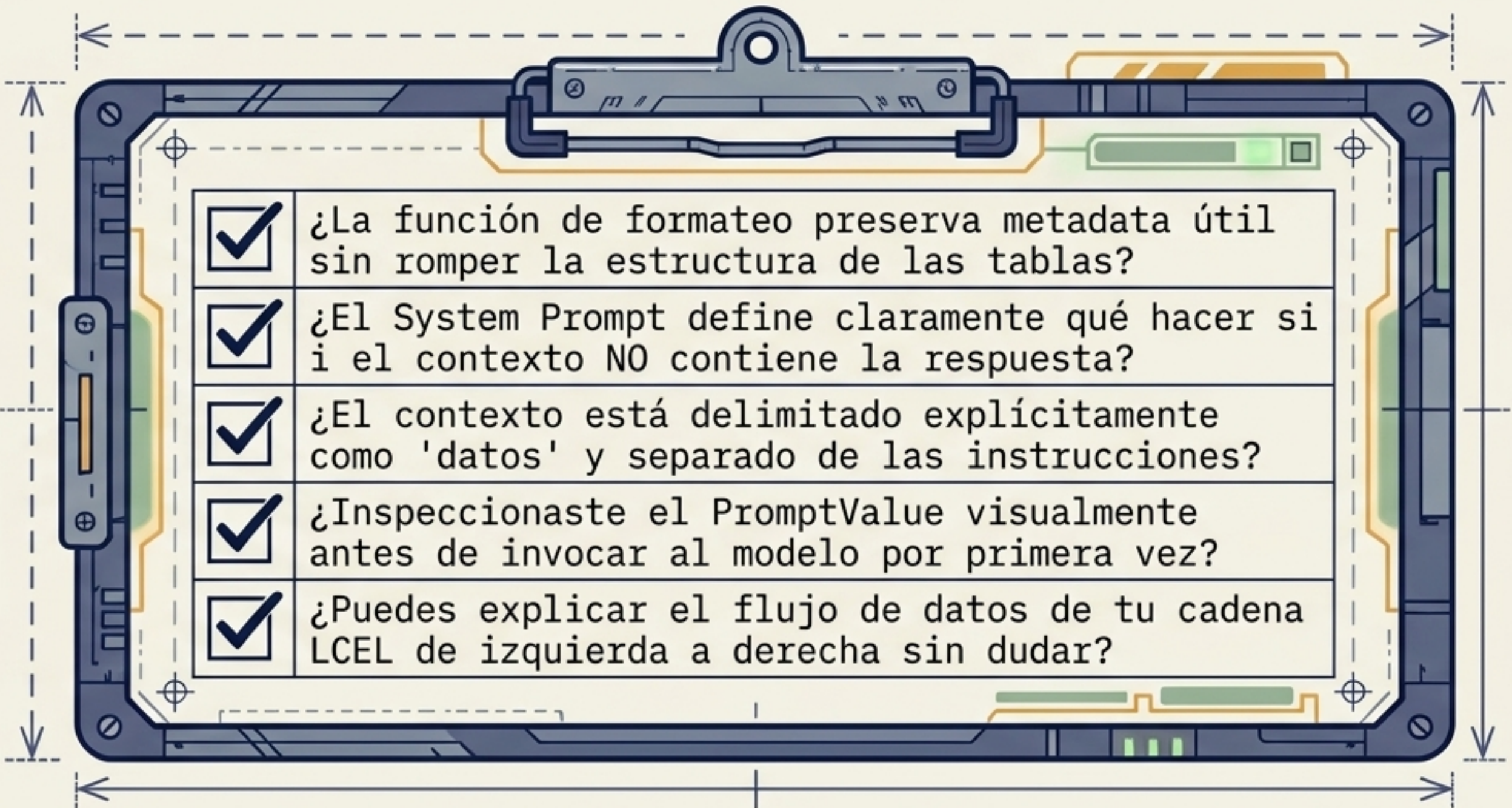
Colocar todo (variables, contexto, reglas) dentro de un gigantesco System Prompt.

REALIDAD



Separación estricta de responsabilidades. Las reglas van al System; el contexto y la pregunta van al **Human Message**.

Checklist de Síntesis Post-Retrieval



☒	¿La función de formateo preserva metadata útil sin romper la estructura de las tablas?
☒	¿El System Prompt define claramente qué hacer si el contexto NO contiene la respuesta?
☒	¿El contexto está delimitado explícitamente como 'datos' y separado de las instrucciones?
☒	¿Inspeccionaste el PromptValue visualmente antes de invocar al modelo por primera vez?
☒	¿Puedes explicar el flujo de datos de tu cadena LCEL de izquierda a derecha sin dudar?

El retriever decide QUÉ información traer. El prompt decide CÓMO presentarla. El modelo decide QUÉ generar con ella.